Easy Study Notes

B.Tech AI & DS · R2021 · CBCS

Explained in simple English with examples

Contents

Part A: U21AD502 Application Development

1. Unit I: Web Essentials
2. Unit II: CSS and Client-Side Scripting (JavaScript)
3. Unit III: Server-Side Development with Node.js *
4. Unit IV: Front-End with React *
5. Unit V: Data-Centric Apps with MongoDB *

Part B: Deep Learning (Units III to V)

6. Unit III: Convolutional Neural Networks
7. Unit IV: Recurrent Neural Networks
8. Unit V: Autoencoders and Generative Models

Last pages: Quick revision and likely exam questions

** Your photo shows the full topic list only for Units I and II. Units III to V are written from the Course Outcomes (CO3: Node.js, CO4: React, CO5: MongoDB). Check them against your full syllabus page.*

How to use these notes

■ Blue = definition ■ Green = example ■ Orange = tip / memory trick
■ Red = exam point

PART A · UNIT I: WEB ESSENTIALS

Syllabus: The Internet and World Wide Web, Basic Protocols, Web Essentials (Clients, Servers and Communication), HTTP Request Message, HTTP Response Message, Web Clients, Web Servers, HTML5 Basic Elements.

1. The Internet

> **Definition**
> The **Internet** is a worldwide network of networks. It connects billions of computers, phones and servers so they can share data.

- **History:** It started in 1969 as **ARPANET**, a US defence project. In 1983 it adopted TCP/IP, which became the base of today's Internet.
- No single person or company owns it. Groups like ICANN (names and addresses) and the IETF (standards) manage it.
- Every device on it has an **IP address**, like a house address. Example: `142.250.183.14`
- Services that run on the Internet: the Web, email, file transfer, video calls, online games.

2. The World Wide Web (WWW)

> **Definition**
> The **WWW** is a collection of linked documents (web pages) stored on servers around the world. You open them with a browser over the Internet. It was invented by **Tim Berners-Lee** at CERN in **1989**.

Internet	World Wide Web
A network (the hardware and connections)	A service (pages and links) that runs on the Internet
Like the **roads**	Like the **shops** along the roads
Started in 1969	Started in 1989
Uses many protocols (TCP/IP, FTP, SMTP...)	Mainly uses HTTP/HTTPS

The 3 building blocks of the Web

1. **HTML**: the language used to write pages.
2. **URL** (Uniform Resource Locator): the address of a page.
3. **HTTP**: the rules for moving pages between server and browser.

Parts of a URL

```
https://www.shop.com:443/products/shoes.html?color=red#reviews
  |          |          |         |               |          |
protocol   domain     port      path           query     fragment
(scheme)   (host)              (file)      (extra data) (section of page)
```

3. Basic Protocols

> **Definition**
> A **protocol** is a set of rules that two computers agree on so they can understand each other. Think of it as a shared language plus good manners.

Protocol	Full form	What it does	Port
IP	Internet Protocol	Gives addresses and routes packets to the right computer	–
TCP	Transmission Control Protocol	Reliable delivery: breaks data into packets, numbers them, resends lost ones	–
UDP	User Datagram Protocol	Fast but unreliable (used for video calls and games)	–
HTTP	HyperText Transfer Protocol	Transfers web pages	80
HTTPS	HTTP Secure	HTTP encrypted with SSL/TLS	443
FTP	File Transfer Protocol	Uploads and downloads files	21
SMTP	Simple Mail Transfer Protocol	**Sends** email	25
POP3 / IMAP	Post Office Protocol / Internet Message Access Protocol	**Receives** email	110 / 143
DNS	Domain Name System	Turns a name into an IP address (google.com → 142.250.x.x)	53
Telnet / SSH	Secure Shell	Remote login (SSH is the secure one)	23 / 22

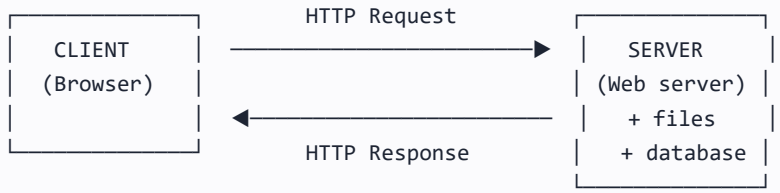
> **Memory trick**
> **S**MTP = **S**end mail. **P**OP = **P**ull mail. DNS = the phone book of the Internet.

TCP/IP model (4 layers)

Layer	Job	Examples
4. Application	What the user's program uses	HTTP, FTP, SMTP, DNS
3. Transport	End-to-end delivery	TCP, UDP
2. Internet	Addressing and routing	IP
1. Network Access	The physical wire or Wi-Fi	Ethernet, Wi-Fi

> **Example: what happens when you type www.google.com and press Enter?**
> 1. The browser asks **DNS**: "What is the IP of www.google.com?" DNS replies with an IP address.
> 2. The browser opens a **TCP** connection to that IP on port 443 (the "3-way handshake": SYN, SYN-ACK, ACK).
> 3. A **TLS** handshake sets up encryption, because it's HTTPS.
> 4. The browser sends an **HTTP request**: `GET / HTTP/1.1`
> 5. Google's server sends an **HTTP response**: `200 OK` plus the HTML.
> 6. The browser reads the HTML, downloads CSS, JS and images, then **renders** (draws) the page.

4. Web Essentials: Clients, Servers and Communication



> **Real-life analogy: a restaurant**
> You (the **client**) look at the menu and place an order (the **request**). The kitchen (the **server**) cooks the food and the waiter brings it to you (the **response**). The waiter is like **HTTP**.

Key points about communication

- The **client always starts** the conversation. The server only answers.
- **HTTP is stateless:** the server forgets you after each request. To remember you (for example, that you're logged in), websites use **cookies**, **sessions** or **tokens**.
- **Request–response cycle:** one request, one response.
- HTTP versions: HTTP/1.0 (one request per connection), HTTP/1.1 (keep-alive connections), HTTP/2 (many requests in parallel), HTTP/3 (runs over QUIC/UDP, faster).

5. HTTP Request Message

This is the message the **browser sends** to the server. It has 4 parts:

```
┌──────────────────────────────────────────────┐
│ 1. Request line : METHOD   URL   VERSION     │
│ 2. Headers      : Name: value (many lines)   │
│ 3. Blank line   : (marks the end of headers) │
│ 4. Body         : data (optional)            │
└──────────────────────────────────────────────┘
```

> **Example: a login form submission**
>
> ```
> POST /login HTTP/1.1                        ← request line
> Host: www.example.com                       ← headers start
> User-Agent: Mozilla/5.0 Chrome/120
> Accept: text/html
> Content-Type: application/x-www-form-urlencoded
> Content-Length: 27
>                                             ← blank line
> username=ravi&password=1234                 ← body
> ```

HTTP methods (request types)

Method	Meaning	Real example
GET	Get (read) data	Open a page, search on Google
POST	Send new data to create something	Sign up, log in, post a comment
PUT	Replace or update the whole thing	Update your full profile
PATCH	Update part of it	Change only your phone number
DELETE	Remove data	Delete a photo
HEAD	Like GET but returns headers only	Check whether a file exists or its size
OPTIONS	Ask which methods are allowed	Used by browsers for CORS checks

Common request headers

- `Host`: which website (required in HTTP/1.1)
- `User-Agent`: which browser or device
- `Accept`: what types the client can handle (text/html, application/json)
- `Content-Type`: the type of data in the body
- `Cookie`: saved cookies sent back to the server
- `Authorization`: login token

Exam point: GET vs POST

GET	POST
Data goes in the URL (`?q=shoes`)	Data goes in the body
Visible, can be bookmarked	Hidden from the URL, can't be bookmarked
Length is limited	No practical limit
Can be cached	Not cached
Use for reading or searching	Use for passwords and forms

6. HTTP Response Message

This is the message the **server sends back**. It also has 4 parts:

```
1. Status line : VERSION  STATUS-CODE  REASON-PHRASE
2. Headers
3. Blank line
4. Body        : the HTML / image / JSON
```

Example

```
HTTP/1.1 200 OK
Date: Sun, 08 Nov 2026 10:00:00 GMT
Server: Apache/2.4
Content-Type: text/html; charset=UTF-8
Content-Length: 52
Set-Cookie: session=abc123

<html><body><h1>Welcome Ravi!</h1></body></html>
```

Status codes (very important)

Range	Meaning	Examples
1xx	Information: "keep going"	100 Continue, 101 Switching Protocols
2xx	**Success ✓**	**200 OK**, 201 Created, 204 No Content
3xx	**Redirect**: "go somewhere else"	301 Moved Permanently, 302 Found, 304 Not Modified (use your cache)
4xx	**Client error**: your mistake	400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (no permission), **404 Not Found**
5xx	**Server error**: the server's mistake	**500 Internal Server Error**, 502 Bad Gateway, 503 Service Unavailable

> **Memory trick**
> 2 = "Yes, here you go". 3 = "Go there". 4 = "**You** messed up". 5 = "**I** messed up".

Common response headers

- `Content-Type`: type of the body (text/html, image/png, application/json)
- `Content-Length`: size in bytes
- `Set-Cookie`: asks the browser to store a cookie
- `Location`: new URL (used with 3xx redirects)
- `Cache-Control`: how long the browser may cache the response

7. Web Clients

> **Definition**
> A **web client** is any software that sends HTTP requests and receives responses. The most common one is the web browser.

- **Examples:** Chrome, Firefox, Edge and Safari (browsers); mobile apps; `curl` and Postman (testing tools); search-engine crawlers.
- **Jobs of a browser:**
  1. Take the URL from the user
  2. Find the IP using DNS
  3. Send the HTTP request
  4. Receive the response
  5. Parse the HTML into the **DOM** and the CSS into the CSSOM, then render the page
  6. Run the JavaScript
  7. Store cookies, cache and history
- **Browser parts:** user interface, rendering engine (Blink, Gecko, WebKit), JavaScript engine (V8, SpiderMonkey), networking, storage.

8. Web Servers

> **Definition**
> A **web server** is software (and the computer it runs on) that waits for HTTP requests and sends back web pages or data.

- **Examples:** Apache HTTP Server, Nginx, Microsoft IIS, Apache Tomcat (for Java), Node.js with Express.
- **How it works:** listen on a port (80 or 443), accept the request, find the file or run code, build the response, send it.

Static web server	Dynamic web server
Sends files exactly as they are stored	Builds the page at request time using code and a database
Everyone sees the same page	The page differs per user
Example: a college "About Us" page	Example: your Instagram feed, your bank balance
Fast and simple	Needs server-side language: Node.js, PHP, Python, Java

Example: a tiny web server in Node.js

```
const http = require('http');
http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<h1>Hello from my server!</h1>');
}).listen(3000);
// Open http://localhost:3000 in the browser
```

9. HTML5: Basic Elements

Definition
HTML (HyperText Markup Language) describes the **structure** of a web page using **tags**. HTML5 is the latest version. It adds semantic tags, audio, video and canvas.

- A **tag** looks like `<p>`. Most tags have an opening and closing pair: `<p>Hello</p>`
- An **element** = opening tag + content + closing tag.
- An **attribute** gives extra information: `<a href="page.html">`
- **Empty (void) tags** have no closing tag: `<br>`, `<img>`, `<hr>`, `<input>`

Basic page structure

```
<!DOCTYPE html>               <!-- tells the browser: this is HTML5 -->
<html lang="en">
  <head>                      <!-- info ABOUT the page (not shown) -->
    <meta charset="UTF-8">
    <title>My First Page</title>
  </head>
  <body>                      <!-- everything you SEE -->
    <h1>Hello World</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

Common tags

Purpose	Tags	Example
Headings	`h1` (biggest) to `h6` (smallest)	`<h1>Title</h1>`
Paragraph and line break	`p`, `br`, `hr`	`<p>Text</p>`
Text formatting	`b` / `strong`, `i` / `em`, `u`, `mark`, `sub`, `sup`	H_2O, x^2
Link	`a`	`<a href="https://google.com">Google</a>`
Image	`img`	`<img src="cat.jpg" alt="A cat" width="200">`
Lists	`ul` (bullets), `ol` (numbers), `li`	see below
Table	`table`, `tr`, `th`, `td`	see below
Containers	`div` (block), `span` (inline)	For grouping and styling

Lists

```
<ul>
  <li>Apple</li>
  <li>Mango</li>
</ul>
<ol>
  <li>Wake up</li>
  <li>Study</li>
</ol>
```

Table

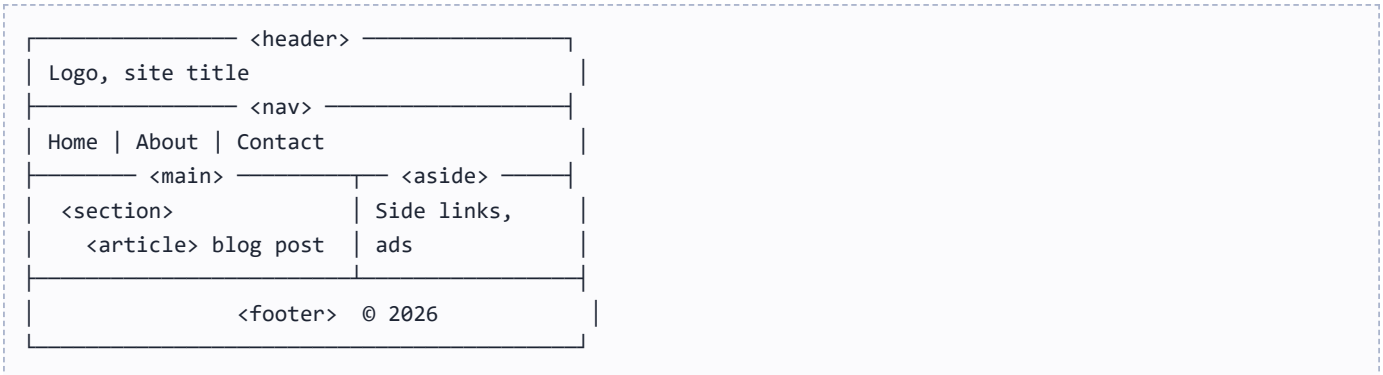
```
<table border="1">
  <tr><th>Name</th><th>Marks</th></tr>
  <tr><td>Ravi</td><td>90</td></tr>
  <tr><td>Priya</td><td>95</td></tr>
</table>
```

Forms (to collect user input)

```
<form action="/register" method="POST">
  Name:  <input type="text" name="name" required>
  Email: <input type="email" name="email">
  Age:   <input type="number" min="18">
  DOB:   <input type="date">
  Gender: <input type="radio" name="g" value="M"> Male
          <input type="radio" name="g" value="F"> Female
  <input type="checkbox"> I agree
  <select name="dept"><option>AI&DS</option><option>CSE</option></select>
  <textarea rows="3"></textarea>
  <button type="submit">Register</button>
</form>
```

New HTML5 input types: `email, number, date, color, range, url, tel, search`. New attributes: `required, placeholder, autofocus, pattern`.

HTML5 semantic elements (they say what the content means)



HTML5 media

```
<video src="movie.mp4" controls width="400"></video>
<audio src="song.mp3" controls></audio>
<canvas id="c" width="200" height="100"></canvas>   <!-- draw with JavaScript -->
```

Exam points

- Block elements (`div, p, h1`) start on a new line and take the full width. Inline elements (`span, a, b, img`) stay in the same line.
- HTML4 vs HTML5: HTML5 adds semantic tags, audio/video, canvas, new input types, local storage, and a simple `<!DOCTYPE html>`.

PART A · UNIT II: CSS AND CLIENT-SIDE SCRIPTING

Syllabus: Introduction to CSS, Basic elements, Inline, Embedded and External style sheets, JavaScript Introduction, JavaScript DOM Model, Using Objects, Regular Expressions.

1. Introduction to CSS

> **Definition**
> **CSS (Cascading Style Sheets)** controls how a page **looks**: colours, fonts, spacing and layout.
> HTML = the **skeleton** (structure). CSS = the **clothes and makeup** (style). JavaScript = the **muscles** (behaviour).

Why "cascading"? When several rules apply to the same element, the browser picks one based on priority, like water flowing down steps.

Advantages: saves time (one CSS file styles the whole site), smaller HTML, consistent look, easy to change, supports different screen sizes.

2. CSS Syntax (basic elements)

```
selector {  property: value;  property: value;  }

h1 { color: blue; font-size: 30px; }
 |      |       |
selector property value
```

Types of selectors

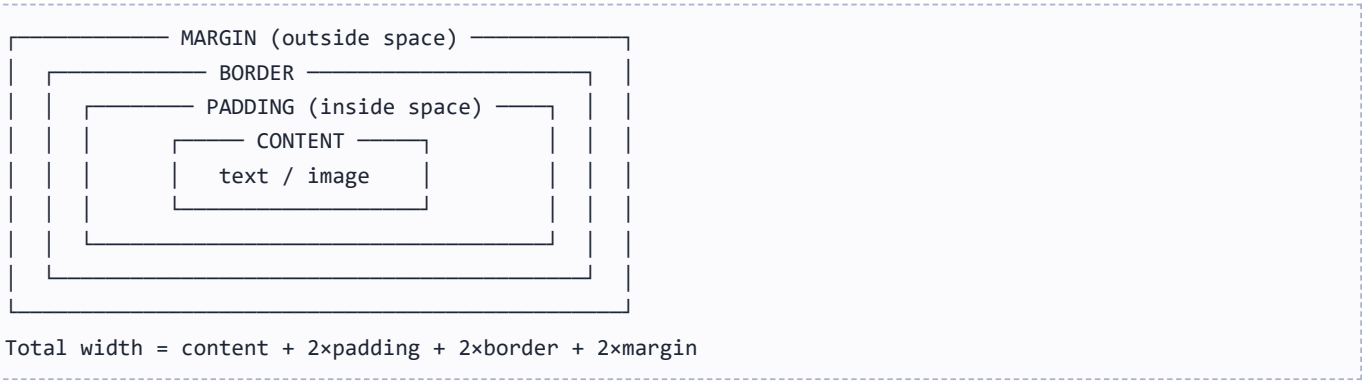
Selector	Syntax	Selects
Element	`p { }`	All <p> tags
Class	`.box { }`	All elements with `class="box"` (can repeat)
ID	`#top { }`	The single element with `id="top"` (must be unique)
Universal	`* { }`	Every element
Group	`h1, h2, p { }`	All of the listed elements
Descendant	`div p { }`	<p> anywhere inside a div
Child	`div > p { }`	<p> directly inside a div
Attribute	`input[type="text"] { }`	Elements with that attribute
Pseudo-class	`a:hover { }`	An element in a special state (hover, first-child, focus)
Pseudo-element	`p::first-letter { }`	Part of an element

Common properties

Group	Properties
Colour and background	`color, background-color, background-image, opacity`
Text and font	`font-family, font-size, font-weight, text-align, text-decoration, line-height`
Box	`width, height, margin, padding, border, border-radius`
Layout	`display (block/inline/flex/grid/none), position, float, z-index`

Colour values: by name (`red`), hex (`#ff0000`), `rgb(255,0,0)`, `rgba(255,0,0,0.5)`. Units: `px` (fixed), `%`, `em/rem` (relative to font size), `vw/vh` (relative to the screen).

The CSS box model (very important)



`Total width = content + 2×padding + 2×border + 2×margin`

> **Example**
> `div { width: 200px; padding: 10px; border: 5px solid black; margin: 20px; }`
> Total width taken = 200 + 20 + 10 + 40 = **270px**

3. The Three Ways to Add CSS

(a) Inline CSS: inside the tag, using the `style` attribute

```
<h1 style="color: red; text-align: center;">Hello</h1>
```

✓ Quick for one element. ✗ Messy and can't be reused.

(b) Embedded / Internal CSS: inside a `<style>` tag in the head

```
<head>
  <style>
    h1 { color: green; }
    .note { background: yellow; }
  </style>
</head>
```

✓ Good for styling a single page. ✗ Can't be shared with other pages.

(c) External CSS: a separate `.css` file, linked in

```
<!-- index.html -->
<head>
  <link rel="stylesheet" href="style.css">
</head>

/* style.css */
body { font-family: Arial; background: #f0f0f0; }
h1   { color: navy; }
```

✓ **Best way:** one file styles the whole website, and the browser caches it. ✗ Needs an extra file download.

> **Exam point: priority (cascading order)**
> **Inline > Embedded/Internal ≈ External**. Between internal and external, whichever comes **later** in the code wins. Specificity: **ID (100) > Class (10) > Element (1)**. Adding `!important` overrides everything.
> Example: if `p {color:red}`, `.x {color:blue}` and `#y {color:green}` all match `<p class="x" id="y">`, the text is **green**.

4. Introduction to JavaScript

> **Definition**
> **JavaScript (JS)** is a scripting language that makes web pages **interactive**: button clicks, form checks, pop-ups and live updates. It runs **inside the browser** (client-side). Brendan Eich created it in 1995.

Note: JavaScript has nothing to do with Java. The names are just similar.

Adding JS to a page

```
<script> alert("Hello!"); </script>          <!-- internal -->
<script src="app.js"></script>                <!-- external (best) -->
<button onclick="alert('Hi')">Click</button>   <!-- inline -->
```

Variables and data types

```
var   a = 10;        // old way, function-scoped
let   b = 20;        // can change, block-scoped
const PI = 3.14;     // cannot change

let name   = "Ravi";            // String
let age    = 20;                // Number
let pass   = true;              // Boolean
let marks  = [90, 85, 70];      // Array
let person = {name:"Ravi", age:20};  // Object
let x;                          // undefined
let y = null;                   // null (empty on purpose)
```

Operators

Arithmetic: `+ - * / % **` Comparison: `== === != < >` Logical: `&& || !`

> **== vs ===**
> `5 == "5"` is **true** because it compares values only. `5 === "5"` is **false** because it also compares the type. Always use `===`.

Conditions and loops

```
if (marks >= 50) { console.log("Pass"); } else { console.log("Fail"); }

for (let i = 1; i <= 5; i++) { console.log(i); }       // prints 1 2 3 4 5

let i = 0;
while (i < 3) { console.log(i); i++; }

switch (day) { case 1: console.log("Mon"); break; default: console.log("?"); }
```

Functions

```
function add(a, b) { return a + b; }        // normal function
const mul = (a, b) => a * b;                 // arrow function
console.log(add(2, 3), mul(2, 3));           // 5 6
```

Pop-up boxes

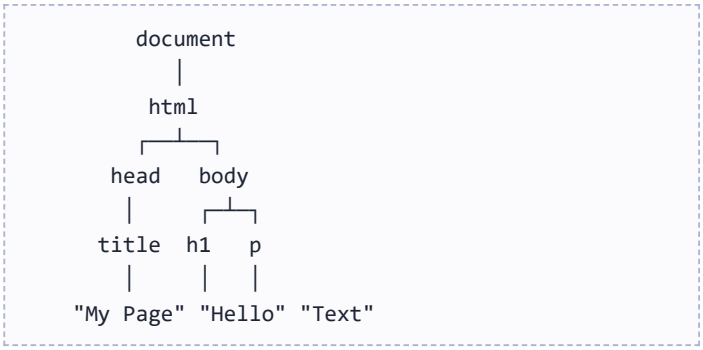
`alert("msg")` shows a message. `confirm("Sure?")` returns true or false. `prompt("Your name?")` returns the text the user types.

5. The JavaScript DOM Model

Definition

The **DOM (Document Object Model)** is a **tree** that the browser builds from the HTML. Every tag becomes a **node** (object). JavaScript uses the DOM to read and change the page while it's open.

```
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>Text</p>
  </body>
</html>
```



Node types: Document node, Element nodes (tags), Text nodes, Attribute nodes. **Relationships:** parent, child and sibling.

Finding (selecting) elements

```
document.getElementById("title")          // one element by id
document.getElementsByClassName("box")    // list of elements by class
document.getElementsByTagName("p")        // list by tag name
document.querySelector(".box")            // first CSS-selector match
document.querySelectorAll("ul li")        // all matches
```

Changing elements

```
let h = document.getElementById("title");
h.innerHTML = "New <b>Title</b>";   // change content (HTML)
h.textContent = "Plain text";          // change text only
h.style.color = "red";                 // change CSS
h.setAttribute("class", "big");        // change an attribute
h.classList.add("active");             // add a class
```

Creating and removing elements

```
let li = document.createElement("li");
li.textContent = "New item";
document.getElementById("list").appendChild(li);   // add
li.remove();                                        // delete
```

Events (reacting to the user)

Common events: `click, dblclick, mouseover, keydown, keyup, submit, change, load, input`

Full example: a click counter

```
<h2 id="count">0</h2>
<button id="btn">Click me</button>
<script>
  let n = 0;
  document.getElementById("btn").addEventListener("click", function () {
    n++;
    document.getElementById("count").textContent = n;
  });
</script>
```

Example: form validation

```
<form onsubmit="return check()">
  <input id="age" type="text"> <button>Submit</button>
</form>
<script>
function check() {
  let a = document.getElementById("age").value;
  if (a === "" || isNaN(a)) { alert("Enter a valid age"); return false; }
  return true;   // form gets submitted
}
</script>
```

6. Using Objects in JavaScript

Definition

An **object** is a collection of **properties** (data) and **methods** (functions) stored as key: value pairs. Think of a **real thing**: a car has properties (colour, brand) and methods (start(), brake()).

Ways to create objects

```
// 1. Object literal (most common)
let student = {
  name: "Ravi",
  age: 20,
  greet: function () { return "Hi, I am " + this.name; }
};
console.log(student.name);       // dot notation      → Ravi
console.log(student["age"]);     // bracket notation  → 20
console.log(student.greet());    // → Hi, I am Ravi
student.dept = "AI&DS";          // add a property
delete student.age;              // remove a property

// 2. Constructor function
function Car(brand, year) { this.brand = brand; this.year = year; }
let c1 = new Car("Tata", 2024);

// 3. Class (ES6)
class Animal {
  constructor(name) { this.name = name; }
  speak() { return this.name + " makes a sound"; }
}
class Dog extends Animal {                // inheritance
  speak() { return this.name + " barks"; }
}
console.log(new Dog("Tommy").speak());     // Tommy barks
```

`this` refers to the object that is calling the method.

Built-in objects

Object	Useful parts
String	`length, toUpperCase(), toLowerCase(), indexOf(), slice(), split(), replace(), trim()`
Array	`length, push(), pop(), shift(), unshift(), sort(), reverse(), map(), filter(), reduce(), forEach()`
Math	`Math.PI, Math.sqrt(), Math.round(), Math.floor(), Math.ceil(), Math.max(), Math.random()`
Date	`new Date(), getDate(), getMonth() (0–11), getFullYear(), getHours()`
Number	`toFixed(2), parseInt(), parseFloat(), isNaN()`
Browser (BOM)	`window, document, navigator, location, history, screen`

```
let nums = [1, 2, 3, 4];
nums.map(x => x * 2);           // [2, 4, 6, 8]
nums.filter(x => x % 2 === 0);  // [2, 4]
nums.reduce((s, x) => s + x, 0);// 10
"hello world".split(" ");        // ["hello", "world"]
Math.floor(Math.random() * 6) + 1;  // dice roll 1–6
```

JSON (JavaScript Object Notation) is the text format used to send objects over HTTP: `JSON.stringify(obj)` turns an object into text. `JSON.parse(text)` turns text back into an object.

7. Regular Expressions (RegEx)

> **Definition**
> A **regular expression** is a **pattern** used to search, match or validate text. Examples: checking whether an email or phone number is in the right format.

```
let re1 = /abc/i;                 // literal form   (i = ignore case)
let re2 = new RegExp("abc", "g"); // constructor form (g = global, find all)
```

Pattern symbols

Symbol	Meaning	Example
`.`	Any single character	`c.t` matches cat, cut
`^`	Start of the string	`^Hi`
`$`	End of the string	`end$`
`[abc]`	Any one of a, b, c	`[aeiou]` = a vowel
`[^abc]`	Anything except a, b, c	
`[a-z]`	A range	`[0-9]` = a digit
`\d \w \s`	Digit, word character (a-z, 0-9, _), whitespace	`\d\d` = two digits
`\D \W \S`	The opposite of the above	
`*`	0 or more	`ab*` matches a, ab, abbb
`+`	1 or more	`\d+`
`?`	0 or 1 (optional)	`colou?r` matches color, colour
`{n}` `{n,m}`	Exactly n times; between n and m times	`\d{10}` = 10 digits
`(...)` and `|`	Group; OR	`(cat|dog)`

Flags: `g` = global (find all), `i` = ignore case, `m` = multiline.

Methods

```
/\d+/.test("abc123")            // true       → does it match?
"abc123".match(/\d+/)[0]        // "123"      → get the match
"a1b2c3".replace(/\d/g, "#")    // "a#b#c#"   → replace
"a,b;c".split(/[,;]/)           // ["a","b","c"]
"hello".search(/l/)             // 2          → position
```

Common validation patterns

```
const mobile  = /^[6-9]\d{9}$/;                    // Indian mobile: 10 digits, starts 6–9
const email   = /^[\w.-]+@[\w-]+\.[a-z]{2,}$/i;    // simple email
const pincode = /^\d{6}$/;                         // 6-digit PIN
const pass    = /^(?=.*[A-Z])(?=.*\d).{8,}$/;      // 8+ chars, 1 capital, 1 digit

console.log(mobile.test("9876543210"));  // true
console.log(email.test("ravi@kpriet.ac.in"));  // true
```

PART A · UNIT III: SERVER-SIDE APPS WITH NODE.JS

Based on CO3: "Develop server side applications using Node.js". Check the exact topics against your full syllabus.

1. What is Node.js?

> **Definition**
> **Node.js** is a runtime that lets JavaScript run **outside the browser**, on the server. It is built on Chrome's **V8 engine**. It was created by Ryan Dahl in 2009.

Features

- **Single-threaded with an event loop:** one thread handles many users by not waiting around.
- **Non-blocking (asynchronous) I/O:** while a file or database is loading, Node serves other requests.
- **Fast** (V8 compiles JS to machine code).
- **npm** (Node Package Manager) gives access to over 2 million free packages.
- The same language (JS) on the front end and back end.

> **Analogy: blocking vs non-blocking**
> **Blocking:** a waiter takes your order and stands in the kitchen until the food is ready. Nobody else gets served.
> **Non-blocking (Node):** the waiter takes your order, gives it to the kitchen, serves other tables, and brings your food when the kitchen calls.

The event loop

```
Request ──▶ Event Queue ──▶ EVENT LOOP (single thread)
                                  │  slow work (file, DB, network)
                                  ▼
                            Thread pool / OS does the work
                                  │  when done → callback
                                  ▼
                            Callback queue ──▶ back to event loop ──▶ Response
```

2. Modules

A **module** is a reusable block of code in its own file. There are three kinds: **core** (built in), **local** (your own) and **third-party** (from npm).

Core module	Use
`http`	Create a web server
`fs`	Read and write files
`path`	Work with file paths
`os`	Operating system information
`events`	Create and handle custom events
`url`	Parse URLs

```
// math.js   (your own module)
exports.add = (a, b) => a + b;

// app.js
const math = require('./math');
console.log(math.add(2, 3));   // 5
```

npm commands: `npm init -y` creates package.json. `npm install express` installs a package. `node app.js` runs a file.

3. File System (fs) Module

```
const fs = require('fs');
// Asynchronous (non-blocking) – preferred
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) return console.log(err);
  console.log(data);
});
// Synchronous (blocking)
const text = fs.readFileSync('data.txt', 'utf8');

fs.writeFile('out.txt', 'Hello', err => {});   // create / overwrite
fs.appendFile('out.txt', ' World', err => {}); // add to the end
fs.unlink('out.txt', err => {});               // delete
```

4. Callbacks, Promises and async/await

```
// Callback
setTimeout(() => console.log("after 1s"), 1000);

// Promise
const p = new Promise((resolve, reject) => { resolve("done"); });
p.then(r => console.log(r)).catch(e => console.log(e));

// async / await (cleanest)
async function load() {
  try {
    const data = await fs.promises.readFile('data.txt', 'utf8');
    console.log(data);
  } catch (e) { console.log(e); }
}
```

> **Callback hell**
> Callbacks nested many levels deep become a "pyramid of doom" that is hard to read. Promises and async/await fix this.

5. The Events Module

```
const EventEmitter = require('events');
const shop = new EventEmitter();
shop.on('order', item => console.log("New order: " + item));  // listener
shop.emit('order', 'Pizza');                                    // fire event
```

6. Express.js: a web framework for Node

Definition

Express is a small, fast framework that makes it easy to build web servers and REST APIs with Node.js.

```
const express = require('express');
const app = express();
app.use(express.json());                    // middleware: read JSON bodies

let students = [{ id: 1, name: "Ravi" }];

app.get('/students', (req, res) => res.json(students));          // READ all
app.get('/students/:id', (req, res) => {                          // READ one
  const s = students.find(x => x.id == req.params.id);
  s ? res.json(s) : res.status(404).send("Not found");
});
app.post('/students', (req, res) => {                             // CREATE
  students.push(req.body); res.status(201).json(req.body);
});
app.delete('/students/:id', (req, res) => {                       // DELETE
  students = students.filter(x => x.id != req.params.id); res.send("Deleted");
});

app.listen(3000, () => console.log("Server on http://localhost:3000"));
```

Key Express ideas

- **Routing:** deciding which code runs for which URL and method (`app.get`, `app.post`...).
- **req** (request): `req.params` (from /:id), `req.query` (from ?x=1), `req.body` (form or JSON data).
- **res** (response): `res.send()`, `res.json()`, `res.status()`, `res.redirect()`.
- **Middleware:** a function that runs **between** the request and the response, used for logging, authentication and parsing. Signature: `(req, res, next) => { ...; next(); }`
- **Static files:** `app.use(express.static('public'))`

REST API (Representational State Transfer)

A way to design web APIs using URLs for **resources** and HTTP methods for **actions**. **CRUD to HTTP:** Create → POST, Read → GET, Update → PUT/PATCH, Delete → DELETE. Data is usually sent as JSON.

PART A · UNIT IV: WEB APPS WITH REACT

Based on CO4: "Implement web applications using React". Check the exact topics against your full syllabus.

1. What is React?

> **Definition**
> **React** is a JavaScript **library**, made by Facebook (Meta) in 2013, for building **user interfaces** out of small reusable pieces called **components**.

- **Component-based:** the UI is built like LEGO blocks (Navbar, Card, Button).
- **Declarative:** you describe *what* the screen should look like, and React works out *how* to update it.
- **Virtual DOM:** makes updates fast.
- **One-way data flow:** data passes from parent to child through props.
- Used for **Single Page Applications (SPAs)**, where the page doesn't fully reload (like Gmail).

Create a project: `npm create vite@latest myapp -- --template react`, then `npm install`, then `npm run dev`. (Older: `npx create-react-app myapp`.)

2. Virtual DOM

```
State changes ─▶ New Virtual DOM ─▶ DIFF with old Virtual DOM ─▶ update ONLY changed parts in the Real DOM
                                    (called "reconciliation")
```

Changing the real DOM is slow. The Virtual DOM is a lightweight copy kept in memory. React compares the old and new copies and updates only the parts that changed.

3. JSX

> **Definition**
> **JSX (JavaScript XML)** lets you write HTML-like code inside JavaScript. Babel converts it into normal JS.

```
const name = "Ravi";
const el = <h1 className="title">Hello, {name}! 2+2 = {2 + 2}</h1>;
```

JSX rules: use `className` instead of `class`. Use `{ }` for JS expressions. Return **one parent element** (or a fragment `<>…</>`). Close every tag (`<img />`). Use camelCase events (`onClick`).

4. Components and Props

```
// Function component (modern way)
function Welcome(props) {
  return <h2>Welcome, {props.name}! Dept: {props.dept}</h2>;
}

function App() {
  return (
    <div>
      <Welcome name="Ravi"  dept="AI&DS" />
      <Welcome name="Priya" dept="CSE" />
    </div>
  );
}
export default App;
```

Props	State
Data passed **into** a component from its parent	Data **owned** by the component
Read-only: can't be changed inside the component	Can change (with a setter function)
Like a function's **arguments**	Like a function's **own variables**

5. State and Hooks

Hooks are special functions that start with `use`. They give function components extra powers.

```
import { useState, useEffect } from 'react';

function Counter() {
  const [count, setCount] = useState(0);          // state variable + setter
  useEffect(() => {                                // runs after render
    document.title = "Clicked " + count + " times";
  }, [count]);                                     // only when count changes
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Hook	Use
`useState`	Store values that change and redraw the UI
`useEffect`	Side effects: fetch data, timers, subscriptions
`useRef`	Access a DOM element directly, or keep a value without redrawing
`useContext`	Share data globally without passing props down many levels

Rules of hooks: call them only at the top level of a component (not inside loops or ifs), and only inside React functions.

6. Events, Lists, Conditional Rendering and Forms

```
function TodoApp() {
  const [items, setItems] = useState([]);
  const [text, setText]   = useState("");

  const add = (e) => {
    e.preventDefault();                      // stop the page reloading
    if (text.trim() === "") return;
    setItems([...items, text]); setText("");
  };

  return (
    <form onSubmit={add}>
      <input value={text} onChange={e => setText(e.target.value)} />   {/* controlled input */}
      <button>Add</button>
      {items.length === 0 ? <p>No tasks yet</p>                          /* conditional */
        : <ul>{items.map((t, i) => <li key={i}>{t}</li>)}</ul>}        {/* list + key */}
    </form>
  );
}
```

- **Keys** help React track which list items changed. Each key must be unique among its siblings.
- **Controlled component:** the input's value is stored in React state.

7. Component Lifecycle

Phase	Meaning	Hook equivalent
Mounting	Component appears on the screen	`useEffect(() => {...}, [])`
Updating	State or props change, so it re-renders	`useEffect(() => {...}, [dep])`
Unmounting	Component is removed	The cleanup function `return () => {...}` inside useEffect

Class components use `componentDidMount`, `componentDidUpdate` and `componentWillUnmount`.

8. Routing and Fetching Data

```
// React Router: switch pages without reloading
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
<BrowserRouter>
  <Link to="/">Home</Link> <Link to="/about">About</Link>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>

// Fetch from your Node/Express API
useEffect(() => {
  fetch("http://localhost:3000/students")
    .then(res => res.json())
    .then(data => setStudents(data));
}, []);
```

PART A · UNIT V: DATA-CENTRIC APPS WITH MONGODB

Based on CO5: "Develop data centric applications using MongoDB". Check the exact topics against your full syllabus.

1. SQL vs NoSQL

SQL (relational), e.g. MySQL	NoSQL (non-relational), e.g. MongoDB
Tables with rows and columns	Documents (JSON-like), key-value, graph
Fixed schema	Flexible schema: each document can differ
Scales vertically (a bigger machine)	Scales horizontally (more machines, "sharding")
Uses joins	Embeds related data inside one document
Good for banking and strict data	Good for big, fast-changing data (social media, IoT)

2. What is MongoDB?

> **Definition**
> **MongoDB** is an open-source **NoSQL document database**. It stores data as **BSON** (Binary JSON) documents.

SQL term	MongoDB term
Database	Database
Table	**Collection**
Row	**Document**
Column	**Field**
Primary key	**_id** (added automatically)

```
// One document in the "students" collection
{
  "_id": ObjectId("6543a1..."),
  "name": "Ravi",
  "age": 20,
  "dept": "AI&DS",
  "skills": ["Python", "React"],              // array
  "address": { "city": "Coimbatore", "pin": 641407 }   // embedded document
}
```

3. CRUD Operations (Mongo shell: mongosh)

```
use college                                  // switch to (or create) a database

// CREATE
db.students.insertOne({ name: "Ravi", age: 20, dept: "AI&DS" })
db.students.insertMany([{ name: "Priya", age: 21 }, { name: "Arun", age: 19 }])

// READ
db.students.find()                           // all documents
db.students.find({ dept: "AI&DS" })          // filter
db.students.findOne({ name: "Ravi" })
db.students.find({ age: { $gt: 19 } })       // age > 19
db.students.find({}, { name: 1, _id: 0 })    // projection: only the name field
db.students.find().sort({ age: -1 }).limit(2) // sort descending, take 2

// UPDATE
db.students.updateOne({ name: "Ravi" }, { $set: { age: 21 } })
db.students.updateMany({}, { $inc: { age: 1 } })   // everyone age + 1

// DELETE
db.students.deleteOne({ name: "Arun" })
db.students.deleteMany({ age: { $lt: 18 } })
```

Query operators

Operator	Meaning	Operator	Meaning
`$eq`	equal	`$in`	in a list
`$ne`	not equal	`$and / $or`	logical
`$gt / $gte`	> / ≥	`$set`	set a field (update)
`$lt / $lte`	< / ≤	`$push`	add to an array (update)

4. Indexing and Aggregation

Index: like the index of a book. It makes searches fast. `db.students.createIndex({ name: 1 })`

Aggregation pipeline: data passes through stages, like water through filters.

```
db.students.aggregate([
  { $match: { age: { $gte: 19 } } },                 // filter
  { $group: { _id: "$dept", total: { $sum: 1 },      // group by dept
              avgAge: { $avg: "$age" } } },
  { $sort:  { total: -1 } }                           // sort
])
```

5. Connecting Node.js to MongoDB (Mongoose)

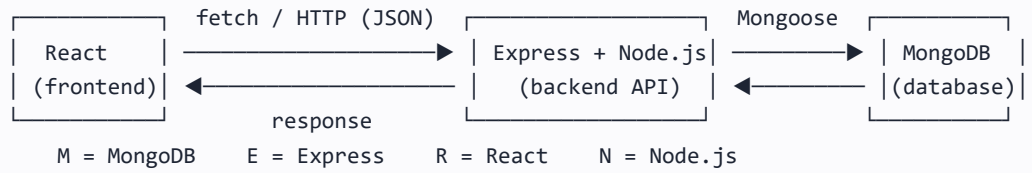
Mongoose is an ODM (Object Data Modeling) library. It adds schemas and validation on top of MongoDB.

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/college');

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age:  { type: Number, min: 17 },
  dept: String
});
const Student = mongoose.model('Student', studentSchema);

// Express routes using the database
app.get('/students', async (req, res) => res.json(await Student.find()));
app.post('/students', async (req, res) => res.status(201).json(await Student.create(req.body)));
app.put('/students/:id', async (req, res) =>
  res.json(await Student.findByIdAndUpdate(req.params.id, req.body, { new: true })));
app.delete('/students/:id', async (req, res) => {
  await Student.findByIdAndDelete(req.params.id); res.send("Deleted"); });
```

6. MERN Stack: how everything connects



PART B · DEEP LEARNING UNIT III: CONVOLUTIONAL NEURAL NETWORKS

Syllabus: Neural Networks and Representation Learning, Convolutional Layers, Implementing the Multichannel Convolution Operation, Train a CNN, Motivation, Pooling, Data Types.

1. Neural Networks and Representation Learning

Definition

A **neural network** is a model made of layers of "neurons". Each neuron computes **output = activation(w·x + b)**. **Representation learning** means the network **learns useful features on its own** from raw data, instead of a human hand-designing them.

Example

To recognise a cat, older methods needed humans to write rules like "detect whiskers". A deep network learns layer by layer: **edges → shapes (eyes, ears) → whole objects (cat face)**.

2. Motivation: why CNNs instead of normal (dense) networks?

- A 224×224 colour image has 224×224×3 = **150,528 inputs**. A fully connected layer with 1000 neurons would need about 150 million weights. That is far too many.
- CNNs use three ideas:
  1. **Sparse (local) connectivity:** each neuron looks at a small patch only (e.g. 3×3).
  2. **Parameter sharing:** the same filter slides over the whole image, so there are far fewer weights.
  3. **Translation equivariance:** a cat is detected wherever it appears in the image.

3. The Convolution Operation

A small matrix called a **filter** (or **kernel**) slides over the image. At each position you multiply element by element and add up the results. The output is called a **feature map**.

Worked example: 4×4 input, 2×2 filter, stride 1

```
Input              Filter          Output (3×3)
1 2 0 1            1 0             1·1+2·0+0·0+1·1 = 2   ...
0 1 3 1      *     0 1      =      2  5  1
1 0 2 2                           0  3  5
2 1 0 1                           2  1  3
```

Top-left: (1×1)+(2×0)+(0×0)+(1×1) = **2**. Then slide one step right and repeat.

Output size formula (very important)

Output = ⌊(N − F + 2P) / S⌋ + 1

N = input size, F = filter size, P = padding, S = stride.

Example: N = 32, F = 5, P = 0, S = 1 gives (32 − 5)/1 + 1 = **28**, so a 28×28 output.

With P = 2: (32 − 5 + 4)/1 + 1 = 32 (the size stays the same; this is called "same" padding).

- **Stride:** how many pixels the filter moves each step. A bigger stride gives a smaller output.
- **Padding:** adding zeros around the border so the edges aren't lost and the size is kept.

4. Multichannel Convolution

A colour image has **3 channels** (R, G, B). So each filter also has 3 channels: its shape is **$F \times F \times C_{in}$**.

- Convolve each channel with its matching filter slice, then **add all the channels together** and add the bias. The result is **1 feature map**.
- Using **K filters** gives **K output channels**.
- Number of parameters = **$(F \times F \times C_{in} + 1) \times K$**

> **Example**
> Input 32×32×3, 10 filters of size 5×5, stride 1, no padding.
> Output = 28×28×10. Parameters = (5×5×3 + 1) × 10 = **760**.

```
# NumPy idea for one output pixel of one filter
out[i, j, k] = np.sum(x[i:i+F, j:j+F, :] * W[:, :, :, k]) + b[k]
```

5. Pooling

> **Definition**
> **Pooling** shrinks the feature map (downsampling). It keeps the important information, reduces computation, and makes the network less sensitive to small shifts.

```
Input 4×4                Max pool 2×2, stride 2     Avg pool 2×2, stride 2
1 3 | 2 4
5 6 | 1 0        ⟶          6   4                     3.75  1.75
----+----
2 2 | 7 8                     3   8                     2.0   6.25
1 3 | 5 5
```

- **Max pooling:** takes the biggest value (most common).
- **Average pooling:** takes the average.
- **Global average pooling:** averages a whole feature map into one number.
- Pooling has **no learnable parameters**.

6. Full CNN Architecture

```
Image ⟶ [Conv ⟶ ReLU ⟶ Pool] ⟶ [Conv ⟶ ReLU ⟶ Pool] ⟶ Flatten ⟶ Dense ⟶ Softmax ⟶ Class
         (feature extraction, repeated)                    (classification)
```

ReLU = max(0, x). It adds non-linearity. **Softmax** turns the outputs into probabilities that add up to 1.

7. Training a CNN

1. Prepare the data: resize, normalise (divide pixel values by 255), split into train, validation and test sets.
2. **Forward pass** to get predictions.
3. **Loss:** cross-entropy for classification.
4. **Backpropagation** computes gradients. An **optimizer** (SGD or Adam) updates the weights.
5. Repeat for many **epochs** in **mini-batches**.
6. Avoid overfitting with **dropout**, **data augmentation** (flip, rotate, crop), **batch normalisation**, early stopping and weight decay.

```
import tensorflow as tf
from tensorflow.keras import layers, models
model = models.Sequential([
  layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
  layers.MaxPooling2D((2,2)),
  layers.Conv2D(64, (3,3), activation='relu'),
  layers.MaxPooling2D((2,2)),
  layers.Flatten(),
  layers.Dense(64, activation='relu'),
  layers.Dropout(0.5),
  layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
```

8. Data Types (for convolution)

Dimensions	Single channel	Multi-channel
1-D	Audio waveform	Skeleton/motion data (many joints over time)
2-D	Grayscale image; audio spectrogram	Colour (RGB) image
3-D	CT/MRI volume scan	Colour video (frames × height × width × RGB)

Famous CNNs: LeNet-5, AlexNet, VGG-16, GoogLeNet (Inception), ResNet (skip connections).

PART B · DEEP LEARNING UNIT IV: RECURRENT NEURAL NETWORKS

Syllabus: Key Limitation: Handling Branching, Automatic Differentiation, Motivation, Recurrent Neural Networks, Implementation of RNNs, RNN Nodes: Vanilla, GRU, LSTM.

1. Key Limitation: Handling Branching

A simple network built as a straight chain of layers (input → layer → layer → output) can't easily handle **branching**: when one value is used in two places, or two paths join together (as in RNNs and ResNets).

When a value feeds several paths, its **gradient is the sum** of the gradients flowing back from every path. Writing that by hand for every model is hard. **Automatic differentiation** solves this.

2. Automatic Differentiation (Autograd)

> **Definition**
> **Automatic differentiation** computes derivatives (gradients) automatically. While the forward pass runs, it records every operation in a **computational graph**. It then applies the **chain rule** backwards through that graph.

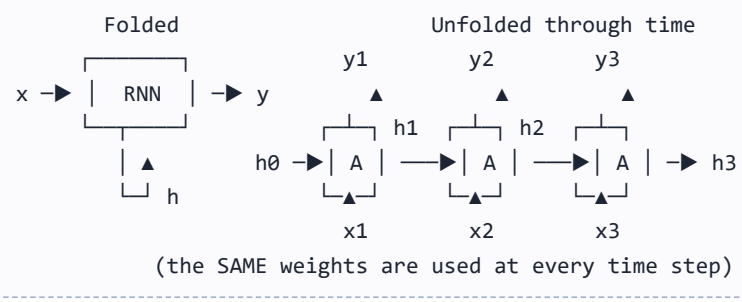
> **Example**
> $f = (a + b) \times b$, with $a = 2$, $b = 3$.
> Forward: $c = a + b = 5$, $f = c \times b = 15$.
> Backward: $\partial f/\partial a = b = 3$. $\partial f/\partial b = c + b = 5 + 3 = \mathbf{8}$. The gradient of b is the **sum** from its two paths: the branching case.

```
import torch
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
f = (a + b) * b
f.backward()
print(a.grad, b.grad)    # tensor(3.) tensor(8.)
```

3. Motivation for RNNs

- Normal networks treat each input separately and have **no memory**.
- Much data is **sequential**: order matters. Examples: sentences, speech, stock prices, weather, music.
- Example: in "I grew up in France… I speak fluent ___", you need to remember "France" to predict "French".
- An RNN keeps a **hidden state (memory)** that is passed from one time step to the next. It can also handle sequences of **different lengths**.

4. Recurrent Neural Network (Vanilla RNN)



Equations

$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$

$y_t = W_{hy} \cdot h_t + b_y$

Types of RNN by input and output

Type	Example
One-to-one	Normal classification
One-to-many	Image captioning (1 image → a sentence)
Many-to-one	Sentiment analysis (a review → positive or negative)
Many-to-many	Machine translation (English → Tamil), video labelling

Training: Backpropagation Through Time (BPTT)

Unfold the RNN over all time steps, then backpropagate. The gradient is multiplied by W_{hh} again and again, which causes:

- **Vanishing gradients:** values shrink toward 0, so the network forgets long-term information. Fix: LSTM or GRU.
- **Exploding gradients:** values blow up. Fix: **gradient clipping**.

5. Implementing an RNN

```
# From scratch (NumPy) – forward pass
h = np.zeros(hidden_size)
for x_t in sequence:
    h = np.tanh(W_xh @ x_t + W_hh @ h + b_h)
    y_t = W_hy @ h + b_y

# Keras
model = tf.keras.Sequential([
  tf.keras.layers.Embedding(10000, 32),
  tf.keras.layers.SimpleRNN(64),          # or LSTM(64) / GRU(64)
  tf.keras.layers.Dense(1, activation='sigmoid')
])
```

6. LSTM (Long Short-Term Memory)

Proposed by Hochreiter and Schmidhuber in 1997. It adds a **cell state (C_t)**, a "conveyor belt" of long-term memory, controlled by **3 gates**. Each gate is a sigmoid that outputs a value from 0 (block) to 1 (allow).

Gate	Question it answers	Formula
Forget gate f_t	What old memory should be thrown away?	$\sigma(W_f[h_{t-1}, x_t] + b_f)$
Input gate i_t	What new information should be stored?	$\sigma(W_i[h_{t-1}, x_t] + b_i)$; candidate $\tilde{C}_t = \tanh(\dots)$
Output gate o_t	What should be output now?	$\sigma(W_o[h_{t-1}, x_t] + b_o)$

Updates

$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad h_t = o_t \odot \tanh(C_t)$

The cell state is updated by **addition**, so gradients flow easily and vanishing is greatly reduced.

> **Analogy**
> Reading a story: the **forget gate** drops details of a minor character who left. The **input gate** notes the new hero's name. The **output gate** decides what's relevant to answer the current question.

7. GRU (Gated Recurrent Unit)

Proposed by Cho et al. in 2014. It's a simpler LSTM with **2 gates** and no separate cell state.

- **Update gate** z_t: how much of the old memory to keep (like forget + input combined).
- **Reset gate** r_t: how much of the past to ignore when computing the new candidate.
- $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

8. Comparison: Vanilla RNN vs LSTM vs GRU

	Vanilla RNN	LSTM	GRU
Gates	0	3 (forget, input, output)	2 (update, reset)
Memory	Hidden state only	Hidden state + cell state	Hidden state only
Long-term memory	Poor (vanishing gradient)	Very good	Good
Parameters / speed	Fewest / fastest	Most / slowest	In between
Use when	Short sequences	Long, complex sequences	Less data, faster training

PART B · DEEP LEARNING UNIT V: AUTOENCODERS AND GENERATIVE MODELS

Syllabus: Regularized Autoencoders, Representational Power, Layer Size and Depth, Stochastic Encoders and Decoders, Denoising, Boltzmann Machines, Deep Belief Networks, Directed Generative Nets.

1. Autoencoder: the basic idea

> **Definition**
> An **autoencoder** is a neural network trained to **copy its input to its output**. It does this through a narrow middle layer called the **code** (or **bottleneck** or latent space). This forces it to learn the most important features. It is **unsupervised**: no labels are needed.

```
Input x ─▶ [ ENCODER f ] ─▶  code h  ─▶ [ DECODER g ] ─▶ Output r ≈ x
 (784)       784→128→32       (32)        32→128→784       (784)

Loss = || x − g(f(x)) ||²   (reconstruction error)
```

> **Analogy**
> Summarising a 10-page chapter into 5 key points (the encoder), then rewriting the chapter from those 5 points (the decoder). If the rewrite is good, your 5 points captured the essence.

Uses: dimensionality reduction, denoising, anomaly detection (a high reconstruction error means unusual data, e.g. fraud), feature learning, image compression.

- **Undercomplete AE:** the code is smaller than the input. It's forced to compress.
- **Overcomplete AE:** the code is equal to or bigger than the input. It might just copy, so it needs **regularisation**.
- A linear AE with MSE loss learns the same subspace as **PCA**.

2. Regularized Autoencoders

Instead of only making the code small, add a penalty to the loss so the AE learns **useful** properties. Loss = reconstruction + Ω(h).

Type	Idea
Sparse AE	Penalty $\lambda \cdot \Sigma \lvert h_i \rvert$: only a few neurons are active at a time
Denoising AE (DAE)	Input is corrupted, output should be the clean version (see below)
Contractive AE (CAE)	Penalises the derivative of h with respect to x, so small input changes don't change the code much (robust features)

3. Representational Power, Layer Size and Depth

- Even a **single-hidden-layer** encoder can in theory represent any mapping (the universal approximation theorem), but it may need a huge number of neurons.
- **Deep autoencoders** (many layers in the encoder and decoder) can:
  - Represent the same function with **far fewer parameters**.
  - Need **less training data**.
  - Give **better compression** than shallow or linear ones.

- A common way to train deep AEs: **greedy layer-wise pretraining**. Train a stack of shallow AEs one at a time, then fine-tune the whole network.
- **Layer size:** too small a code underfits (loses information); too large and it just copies the input.

4. Stochastic Encoders and Decoders

Instead of a fixed output, the encoder and decoder output **probability distributions**:

- Encoder: $p_{encoder}(h \mid x)$. Decoder: $p_{decoder}(x \mid h)$.
- Training minimises $\mathbf{-\log p_{decoder}(x \mid h)}$. For real values (a Gaussian), this is the same as MSE. For binary values (Bernoulli), it's cross-entropy.
- Leads to the **Variational Autoencoder (VAE)**: the encoder outputs a mean μ and spread σ; we **sample** $h = \mu + \sigma \cdot \varepsilon$ (the reparameterisation trick). Loss = reconstruction + KL divergence. We can then **generate new data** by sampling h and decoding it.

5. Denoising Autoencoders

```
clean x ─▶ add noise ─▶ x̃ (corrupted) ─▶ Encoder ─▶ h ─▶ Decoder ─▶ r      Loss = || x − r ||²
                                                                        (compare with CLEAN x)
```

- Noise types: Gaussian noise, masking (randomly set pixels to 0), salt-and-pepper.
- The network can't simply copy, so it must learn the **real structure** of the data.
- Example: cleaning noisy scanned documents or old photos.

```
noisy = x_train + 0.3 * np.random.normal(size=x_train.shape)
autoencoder.fit(noisy, x_train, epochs=10)    # input = noisy, target = clean
```

6. Boltzmann Machines

> **Definition**
> A **Boltzmann Machine (BM)** is an **energy-based**, undirected, stochastic network of binary units. Low-energy states are more likely. $P(x) = e^{-E(x)} / Z$, where Z is the normalising constant (the partition function).

- It has **visible units** v (the data) and **hidden units** h (features).
- In a **full BM** every unit connects to every other unit, which makes it very slow to train.

Restricted Boltzmann Machine (RBM)

```
  Hidden:   (h1)  (h2)  (h3)
              | \  / | \ / |       connections ONLY between layers
              |  ╳   |  ╳  |       NO visible–visible or hidden–hidden links
  Visible:  (v1)  (v2)  (v3)  (v4)
```

- Energy: $E(v, h) = -b^{T}v - c^{T}h - v^{T}Wh$
- Because there are no links within a layer, the hidden units are independent given v, which makes sampling easy.
- Trained with **Contrastive Divergence (CD-k)** using Gibbs sampling: $v \to h \to v' \to h'$, then update $W \propto (v \cdot h^{T})_{data} - (v' \cdot h'^{T})_{model}$.
- Uses: recommender systems (the Netflix prize), feature learning, pretraining.

7. Deep Belief Networks (DBN)

- Proposed by Geoffrey Hinton in 2006. This kick-started modern deep learning.

- Built by **stacking RBMs**. The hidden layer of one RBM becomes the visible layer of the next.
- **Structure:** the top two layers are **undirected** (an RBM). The lower layers are **directed** (pointing down toward the data).
- **Training:**
  1. **Greedy layer-wise pretraining** (unsupervised): train RBM 1, freeze it, train RBM 2 on its outputs, and so on.
  2. **Fine-tuning** (supervised): add an output layer and train everything with backpropagation.

```
Top RBM  (undirected)  h3 ═══ h2
                            ↓     directed
                            h1
                            ↓     directed
                            v  (data)
```

8. Directed Generative Nets

Models that **generate new data** by sampling a random code z and passing it through a network: z → network → x.

Model	Main idea
Sigmoid Belief Networks	Directed graph of binary sigmoid units; each layer generates the one below
Differentiable Generator Networks	A neural net g(z) turns simple noise (Gaussian) into complex data
Variational Autoencoder (VAE)	Encoder + decoder trained to maximise the ELBO (reconstruction − KL); smooth latent space
Generative Adversarial Network (GAN)	Two networks compete: the **Generator** makes fakes, the **Discriminator** spots fakes. Both improve until the fakes look real.
Autoregressive nets (PixelCNN, WaveNet)	Generate one piece at a time: $p(x) = \Pi\, p(x_i \mid x_{1..i-1})$

GAN analogy

The **generator** is a forger making fake currency notes. The **discriminator** is a police officer checking them. As the police get better, the forger improves, until the fakes can't be told apart. Used for realistic faces (StyleGAN), image-to-image translation and super-resolution.

GAN loss (minimax game)

$\min_G \max_D \quad E[\log D(x)] + E[\log(1 - D(G(z)))]$

QUICK REVISION AND LIKELY EXAM QUESTIONS

Application Development: one-line facts

Topic	Remember
WWW inventor	Tim Berners-Lee, 1989, at CERN
HTTP / HTTPS ports	80 / 443
DNS	Name → IP address
SMTP vs POP3/IMAP	Send vs receive email
HTTP request parts	Request line, headers, blank line, body
HTTP response parts	Status line, headers, blank line, body
404 / 500	Not Found (client) / Internal Server Error (server)
CSS priority	Inline > Internal/External (the later one wins); ID > class > element
Box model	Content → padding → border → margin
DOM	A tree of objects built from HTML; JS changes it
== vs ===	Value only vs value AND type
Regex `^ $ \d + * ? {n}`	Start, end, digit, 1+, 0+, 0 or 1, exactly n
Node.js	JS on the server, V8, event loop, non-blocking
React	Components, JSX, props (read-only), state (useState), Virtual DOM
MongoDB	Collection = table, document = row, BSON, _id

Deep Learning: one-line facts

Topic	Remember
Conv output size	$(N - F + 2P)/S + 1$
Conv parameters	$(F \cdot F \cdot C_{in} + 1) \cdot K$
Why CNNs	Sparse connections, parameter sharing, translation equivariance
Pooling	Downsamples; max or average; no parameters
Autodiff	Computational graph + chain rule; branching means gradients add up
RNN problem	Vanishing/exploding gradients (fix: LSTM/GRU, gradient clipping)
LSTM gates	Forget, input, output + cell state
GRU gates	Update, reset
Autoencoder	Encoder → bottleneck → decoder; minimise reconstruction error
Denoising AE	Noisy input → clean output
RBM	Visible + hidden layers, no links within a layer, trained with Contrastive Divergence
DBN	Stacked RBMs, greedy layer-wise training (Hinton, 2006)
GAN	Generator vs discriminator (minimax)

Likely 2-mark questions

1. Differentiate the Internet and the WWW.
2. What is a protocol? Name any four.
3. List any four HTTP status codes with their meaning.
4. Differentiate GET and POST.
5. What are semantic elements in HTML5? Give examples.
6. What is the CSS box model?
7. What is the DOM?
8. Write a regular expression to validate a 10-digit mobile number.
9. What is the event loop in Node.js?
10. Differentiate props and state in React.
11. What is pooling? Why is it used?
12. What is the vanishing gradient problem?
13. What is a denoising autoencoder?
14. Define an RBM.

Likely 13/16-mark questions

1. Explain the structure of HTTP request and response messages with examples.
2. Explain the client–server architecture and the working of web clients and web servers.
3. Explain the three types of CSS with examples and their priority.
4. Explain the JavaScript DOM model. Write a program to change content and style on a button click.
5. Explain JavaScript objects (creation, properties, methods, built-in objects) with examples.
6. Explain regular expressions in JavaScript with a form validation example.
7. Develop a REST API using Node.js and Express for CRUD operations.
8. Explain React components, props, state and hooks with an example.
9. Explain CRUD operations in MongoDB with examples.
10. Explain the CNN architecture with convolution, multichannel convolution and pooling. Solve an output-size problem.
11. Explain the RNN and its training. Compare it with LSTM and GRU using diagrams.
12. Explain autoencoders and their types (regularized, denoising, stochastic).
13. Explain Boltzmann machines and Deep Belief Networks.
14. Explain directed generative networks, including the VAE and GAN.

> **Exam writing tips**
> Start with a **definition**, then a **diagram**, then the **explanation in points**, then an **example or code**, then **advantages and applications**. Diagrams and examples score extra marks.

— All the best for your exams! —